

CPCS 203 – Object-Oriented Programming

Assignment 2: Hospital Management System

Abstract Classes, Interfaces & Inheritance

Department of Computer Science

Second Semester, 2025/2026

Abstract

This assignment requires you to apply the core principles of Object-Oriented Programming — **abstract classes**, **interfaces**, **inheritance**, and **polymorphism** — in a realistic context: a Hospital Management System. You are provided with `Main.java` (already complete), the expected `input.txt`, and the expected `output.txt`. Your task is to implement **all remaining classes from scratch** so that when you run `Main.java`, it reads commands from `input.txt` and produces an `output.txt` that matches the expected output.

1. Introduction

A hospital management system is a software application that organises the daily operations of a hospital, including managing medical staff, patients, prescriptions, and surgical operations. In this assignment you will build a simplified version of such a system.

The system manages three types of medical staff — **Doctors**, **Nurses**, and **Surgeons** — each with their own responsibilities. Not all staff can prescribe medication, and only surgeons can perform operations. These differences are modelled using **interfaces**.

2. System Flow

The following steps describe the complete lifecycle of the system. Read them carefully before you start coding.

Step 1. Register Medical Staff.

The administrator adds doctors, nurses, and surgeons to the system. Each staff member has a unique ID, name, and salary. Each type has additional specific attributes (e.g. specialty for doctors, shift for nurses, surgery type for surgeons).

Step 2. Register Patients.

The administrator adds patients to the system. Each patient has a unique ID, name, age, and a diagnosis.

Step 3. Assign Staff to Patient.

A staff member is assigned to a patient, creating a `MedicalRecord` that tracks the treatment and its status.

Step 4. Prescribe Medication.

Only staff that implement the `Prescribable` interface (i.e. `Doctor` and `Surgeon`) may prescribe medication for a patient.

- If the staff member is `Prescribable`: the prescription is recorded and a success message is printed.
- If not (e.g. a `Nurse`): an error message is printed.

Step 5. Schedule an Operation.

Only staff that implement the `Operable` interface (i.e. `Surgeon`) may schedule an operation for a patient. The operation is recorded with a scheduled date and status *“Scheduled”*.

Step 6. Complete an Operation.

The surgeon completes the scheduled operation. The status of the corresponding record is updated to *“Completed”*.

Step 7. Display Information.

The system can display all staff members, all patients, and the full medical records of a specific patient.

3. OOP Concepts

This assignment is designed to practise the following OOP concepts. Read this section carefully — it explains *why* each concept is used in the system.

- **Abstract Class** — `MedicalStaff`.

All staff share common attributes (ID, name, salary) and a common abstract method `getDuties()`. Because different types of staff have different duties, this method *cannot* be implemented in the base class — each subclass must provide its own implementation. This enforces a contract without providing a default behaviour.

- **Inheritance**.

`Doctor`, `Nurse`, and `Surgeon` all **extend** `MedicalStaff`. They inherit the common fields and each adds its own specific attributes and overrides `getDuties()`.

- **Interface 1** — `Prescribable`.

Defines the ability to prescribe medication. Implemented by `Doctor` and `Surgeon`. `Nurse` does *not* implement it, reflecting the real-world rule that nurses cannot prescribe.

- **Interface 2 — Operable.**

Defines the ability to perform surgical operations. Implemented *only* by **Surgeon**. This demonstrates that a class can implement **multiple interfaces** simultaneously.

- **Polymorphism.**

The system stores all staff in a single **MedicalStaff[]** array. At runtime, **instanceof** checks determine whether a staff member is **Prescribable** or **Operable** before calling the corresponding interface methods.

Important Note

Key insight about interfaces.

A class can implement *multiple* interfaces but extend only *one* class. **Surgeon** extends **MedicalStaff** and simultaneously implements both **Prescribable** *and* **Operable**. This is the power of interfaces: they allow unrelated capabilities to be added to a class independently.

4. System Entities

The system manages the following entities:

Entity	Description
MedicalStaff	Abstract base class for all hospital staff. Holds staffId , name , and salary . Declares the abstract method getDuties() .
Doctor	Extends MedicalStaff , implements Prescribable . Has specialty and years of experience .
Nurse	Extends MedicalStaff . Has shift type and department . Does <i>not</i> implement any interface.
Surgeon	Extends MedicalStaff , implements both Prescribable and Operable . Has surgery type and operating room number .
Patient	A patient registered in the hospital. Has patientId , name , age , and diagnosis .
MedicalRecord	Created when a staff member is assigned to a patient. Stores the staff , patient , treatment details , and status .

The two interfaces used by the system are:

Interface	Implemented by	Methods
Prescribable	Doctor , Surgeon	prescribeMedication(Patient, String) , getPrescriptions()
Operable	Surgeon only	scheduleOperation(Patient, String) , completeOperation(Patient)

5. What Is Provided

The following items are provided to you. **Everything else must be implemented from scratch.**

Provided: Main.java

The entry point of the system. It contains the `main` method, which is **already fully implemented** — do not modify it. It reads commands from `input.txt`, calls your `HospitalSystem` methods, and writes results to `output.txt`. Think of it as a test driver: if your implementation is correct, simply running `Main.java` will produce the expected output.

Provided: input.txt

The input file containing all commands the system must execute. Place it in the **project root directory** (same level as the `src` folder).

Provided: output.txt (expected)

The expected output your program must produce. Use it to verify that your implementation is correct.

6. UML Class Diagram

Figure 1 shows the complete class diagram for the system. Study it carefully before you begin coding. **Every class and interface shown in the diagram must be implemented by you from scratch.**

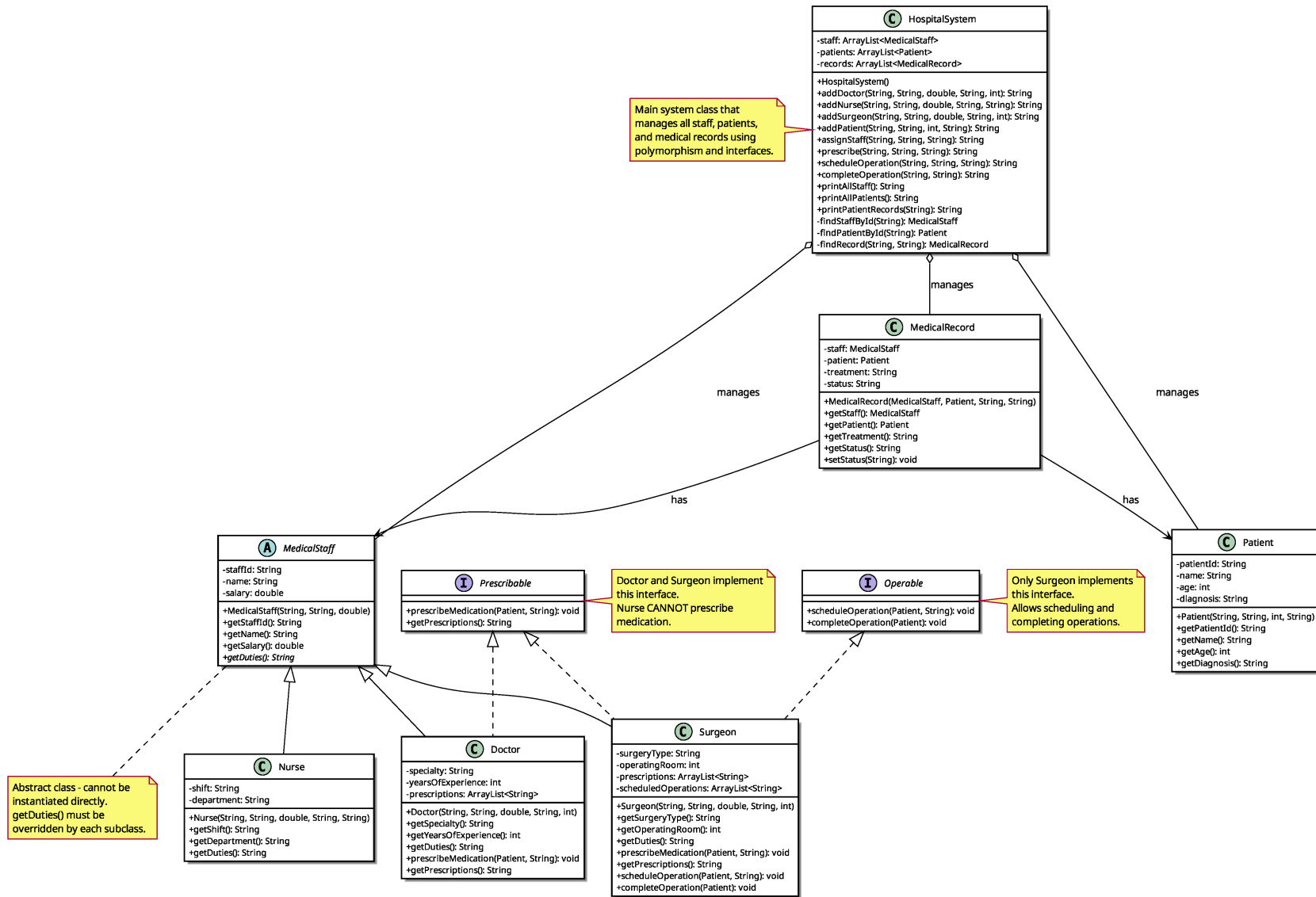


Figure 1: Hospital Management System — UML Class Diagram

7. Student Tasks

You must implement **nine files** from scratch. Each task below uses a table to show exactly what you need to implement.

Task 1 — Prescribable.java (interface)

Type	interface
Method 1	void prescribeMedication(Patient patient, String medication) Prescribes a medication for the given patient.
Method 2	String getPrescriptions() Returns a formatted string listing all prescriptions issued.

Task 2 — Operable.java (interface)

Type	interface
Method 1	void scheduleOperation(Patient patient, String date) Schedules a surgical operation for the given patient.
Method 2	void completeOperation(Patient patient) Marks the operation for the given patient as completed.

Task 3 — MedicalStaff.java (abstract class)

Type	abstract class
Fields	staffId (String), name (String), salary (double)
Constructor	Initialise all three fields.
Getters	getStaffId(), getName(), getSalary()
Abstract	String getDuties() — must be overridden by every subclass.

Task 4 — Doctor.java (extends MedicalStaff, implements Prescribable)

Type	class extends MedicalStaff implements Prescribable
Extra fields	specialty (String), yearsOfExperience (int)
Constructor	Call <code>super()</code> , then initialise the two extra fields.
Override	<code>getDuties()</code> — return a string describing a doctor's duties.
Implement 1	<code>prescribeMedication(Patient patient, String medication)</code> Store the prescription internally and return a confirmation.
Implement 2	<code>getPrescriptions()</code> — return a formatted list of all prescriptions.

Task 5 — Nurse.java (extends MedicalStaff only)

Type	class extends MedicalStaff
Extra fields	shift (String — e.g. <i>Morning</i> / <i>Evening</i> / <i>Night</i>), department (String)
Constructor	Call <code>super()</code> , then initialise the two extra fields.
Override	<code>getDuties()</code> — return a string describing a nurse's duties.

Task 6 — Surgeon.java (extends MedicalStaff, implements Prescribable & Operable)

Type	class extends MedicalStaff implements Prescribable, Operable
Extra fields	surgeryType (String), operatingRoom (int)
Constructor	Call <code>super()</code> , then initialise the two extra fields.
Override	<code>getDuties()</code> — return a string describing a surgeon's duties.
Implement 1	<code>prescribeMedication(Patient patient, String medication)</code> Same logic as Doctor: store and return confirmation.
Implement 2	<code>getPrescriptions()</code> — return a formatted list of all prescriptions.
Implement 3	<code>scheduleOperation(Patient patient, String date)</code> Record the operation with status <i>Scheduled</i> and return confirmation.
Implement 4	<code>completeOperation(Patient patient)</code> Update the operation status to <i>Completed</i> and return confirmation.

Task 7 — Patient.java (data class)

Type	class
Fields	patientId (String), name (String), age (int), diagnosis (String)
Constructor	Initialise all four fields.
Getters	getPatientId(), getName(), getAge(), getDiagnosis()

Task 8 — MedicalRecord.java (data class)

Type	class
Fields	staff (MedicalStaff), patient (Patient), treatment (String), status (String)
Constructor	Initialise all four fields.
Getters	getStaff(), getPatient(), getTreatment(), getStatus()
Setter	setStatus(String status)

Task 9 — HospitalSystem.java (controller class)

Type	class
Fields	Three ArrayList collections: one for staff, one for patients, one for records.
Constructor	Initialise all three lists.

Methods to implement — each must return a String that Main.java writes to output.txt:

-
- #1 `addDoctor(staffId, name, salary, specialty, yearsOfExperience)`
Create a `Doctor` and add it to the staff list.
 - #2 `addNurse(staffId, name, salary, shift, department)`
Create a `Nurse` and add it to the staff list.
 - #3 `addSurgeon(staffId, name, salary, surgeryType, operatingRoom)`
Create a `Surgeon` and add it to the staff list.
 - #4 `addPatient(patientId, name, age, diagnosis)`
Create a `Patient` and add it to the patients list.
 - #5 `assignStaff(staffId, patientId, treatment)`
Look up staff and patient by ID; create a `MedicalRecord` with status *Active*.
 - #6 `prescribe(staffId, patientId, medication)`
Check staff implements `Prescribable`; return an error if not.
 - #7 `scheduleOperation(staffId, patientId, date)`
Check staff implements `Operable`; return an error if not.
 - #8 `completeOperation(staffId, patientId)`
Update the matching record status to *Completed*.
 - #9 `printAllStaff()`
Return a formatted table of all registered staff.
 - #10 `printAllPatients()`
Return a formatted table of all registered patients.
 - #11 `printPatientRecords(patientId)`
Return all medical records for the specified patient.
-

Important Note

Main.java is provided — do not implement it yourself.

`Main.java` is given to you as part of this assignment. It contains the `main` method, which is **already complete**. Your only required change is to fill in the two constants at the top of the class body: `static final String STUDENT_ID` and `static final String STUDENT_NAME` (see Submission Instructions). Do not modify anything else in `Main.java`. It is provided solely to *use* your implementation — if all nine classes are correctly implemented, running `Main.java` will automatically produce the correct output.

8. Input and Output Files

The program reads commands from `input.txt` and writes all output to `output.txt`. Both files must be placed in the **project root directory** (the same level as the `src` folder).

8.1. Input File Format

Each line in `input.txt` contains one command. Lines starting with `#` are comments and are ignored. Names with spaces use an underscore (e.g. `Ahmed_Ali`).

Command	Description
<code>ADD_DOCTOR id name salary specialty years</code>	Register a doctor.
<code>ADD_NURSE id name salary shift department</code>	Register a nurse.
<code>ADD_SURGEON id name salary surgeryType room</code>	Register a surgeon.
<code>ADD_PATIENT id name age diagnosis</code>	Register a patient.
<code>ASSIGN staffId patientId treatment</code>	Assign a staff member to a patient.
<code>PRESCRIBE staffId patientId medication</code>	Prescribe medication (only if Prescribable).
<code>SCHEDULE_OP staffId patientId date</code>	Schedule an operation (only if Operable).
<code>COMPLETE_OP staffId patientId</code>	Complete a scheduled operation.
<code>DISPLAY_STAFF</code>	Print all registered staff members.
<code>DISPLAY_PATIENTS</code>	Print all registered patients.
<code>DISPLAY_RECORDS patientId</code>	Print all medical records for a specific patient.

9. How to Start

Follow these steps **in order** before you write a single line of code.

Step 1. Understand What Is Provided.

You have been given a complete `Main.java`. Open it and read through it — the main method is **already done for you**.

- **Do not modify** `Main.java` (except to fill in your `STUDENT_ID` and `STUDENT_NAME`).
- It exists purely to *use* your implementation. It reads commands from `input.txt`, calls the appropriate `HospitalSystem` methods, and writes every returned `String` to `output.txt`.
- If you correctly implement all required classes, simply running `Main.java` will produce the expected output — no further changes needed.

Step 2. Read the Full Specification.

Read everything — introduction, system flow, OOP concepts, and all task descriptions. You must understand the purpose of every class and method *before* you open your IDE.

Step 3. Study the UML Diagram.

The UML diagram (Section 6) shows how every class and interface fits together. Trace the inheritance hierarchy, identify which classes implement which interfaces, and understand the relationships between objects.

Step 4. Understand the Abstraction Level.

The `HospitalSystem` class is the **core of your application**. Its public methods — `addDoctor`, `addSurgeon`, `addPatient`, `assignStaff`, and the rest — define the *required contract* your system must honour. These method signatures **must be implemented exactly as specified**; they are non-negotiable.

Step 5. You Have Design Freedom — Use It Wisely.

Beyond the required abstraction level, you are free to design your internal implementation however you like, as long as it follows OOP principles. The evaluator will not penalise creative internal design choices — what matters is that your system produces the correct output.

Step 6. Focus on Functionality.

Before polishing your code, make sure every required method produces the correct output. Use the provided `input.txt` and `output.txt` to test your implementation continuously.

10. Submission Instructions

Warning – Required

Student ID and Name are mandatory.

Your `Main.java` **must** include the following two constant declarations at the very top of the class body, filled in with your own information. Submissions that are missing these declarations, or that still contain the placeholder values, **will not be graded**.

```
public class Main {
    // — TODO: Write your Student ID and Name here —
    static final String STUDENT_ID   = "YOUR_ID_HERE";
    static final String STUDENT_NAME = "YOUR_NAME_HERE";
    // —
    public static void main(String[] args) {
```

1. Implement all required files from scratch:
 - `Prescribable.java`, `Operable.java`
 - `MedicalStaff.java`, `Doctor.java`, `Nurse.java`, `Surgeon.java`
 - `Patient.java`, `MedicalRecord.java`, `HospitalSystem.java`
2. In the **provided** `Main.java`, fill in your `STUDENT_ID` and `STUDENT_NAME` as shown above. Do *not* modify anything else in that file.
3. Run your program with the provided `input.txt` and verify that your `output.txt` matches the expected output.

4. Submit the **entire project folder** as a zip file.
5. Name your zip file: `StudentID_StudentName_Assignment2.zip`
Example: `4410012345_Ahmed_Ali_Assignment2.zip`

Important Note

Academic Integrity. All submitted work must be your own. You may discuss ideas with classmates, but you must write your own code independently. Any form of copying will result in a zero grade for all parties involved.